CROP DISEASE MANAGEMENT SYSTEM

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Parteek - EN23CS301715

Madhav - EN24CS3L10012

Parth gupta - EN23CS301716

Under the Guidance of

Prof. Trishna

Prof. Devendra singh bais

Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE - 453331

APRIL - 2026

Report Approval

The project work **"Crop Disease Management System"** is hereby approved as a creditable study of an engineering subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approved any statement made, opinion expressed, or conclusion drawn there in; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner

Name:

Designation:

Affiliation:

External Examiner

Name:

Designation

Affiliation

Declaration

We hereby declare that the project entitled **"Crop Disease Management System"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology / Master of Computer Applications in ' Computer Science and Engineering' completed under the supervision of **Dr. Trishna Professor, and Dr. Devendra singh bias**, Professor, Faculty of Engineering, MediCaps University Indore is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student(s) with date

Parteek(EN23CS301715) _____

Madhav(EN24CS3L10012) _____

Parth Gupta(EN23CS301716) _____

Certificate

We, **Dr. Trishna** , Dr. **Devendra singh bias** certify that the project entitled **"Crop Disease Management System"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology by **Parteek [EN24CS301715] , Madhav [EN24CS3L10012], Parth gupta**

[EN23CS301716] is the record carried out by them under our guidance and that the work has not formed the basis of award of any other degree elsewhere.

_____

_____

Prof. Trishna
Computer Science and Engineering
MediCaps University

Prof. Devendra singh bais
Computer Science and Engineering
MediCaps University

_____

Dr. Kailash Chandra Bandhu
Head of the Department
Computer Science & Engineering
MediCaps University, Indore

Acknowledgements

I would like to express my deepest gratitude to Honorable Chancellor, **Shri R C Mittal**, who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D K Patnaik,** Vice Chancellor, Medi-Caps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank Prof. **(Dr.) Ratnesh Litoriya,** Associate Dean, Faculty of Engineering, Medi-Caps University, for giving me a chance to work on this project. I would also like to thank my Head of the Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

I express my heartfelt gratitude to Internal Guide, **Dr. Trishna and Dr. Devendra singh bais,** Professor, Department of Computer Science Engineering, MU, without whose continuous help and support, this project would ever have reached to the completion.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

Parteek [EN23CS301715] ,Madhav [EN24CS3L10012] ,
Parth gupta[EN23CS301716]

B.Tech. IV Year
Department of Computer Science & Engineering
Faculty of Engineering
MediCaps University, Indore

Abstract

Crop diseases pose a significant threat to agricultural productivity, leading to reduced yield and substantial economic losses for farmers. Early identification and timely management of plant diseases are essential for ensuring healthy crop growth, yet traditional diagnosis methods often require expert knowledge and laboratory analysis, which are not always accessible to farmers. To address this challenge, this study presents a Crop Disease Management System that utilizes machine learning and mobile technology to provide an efficient, accurate, and user-friendly solution for disease detection.

The system integrates a Convolutional Neural Network (CNN) model developed using Keras and TensorFlow for classifying crop leaf diseases based on images. A Flutter-based mobile application serves as the user interface, allowing farmers to upload leaf images and instantly receive disease predictions along with treatment and preventive recommendations. The trained model achieves high accuracy, while the integration of TensorFlow Lite ensures fast, offline predictions suitable for rural areas with limited internet access. User feedback confirms that the system is easy to navigate, practical for field use, and helpful in supporting informed decisionmaking.

Overall, the system demonstrates how artificial intelligence and mobile applications can significantly enhance modern farming practices. The study contributes a reliable, scalable, and farmer-centric solution that can reduce expert dependency, minimize crop loss, and promote sustainable agriculture. With future enhancements such as multi-language support, expanded datasets, and IoT integration, the system holds strong potential for large-scale deployment in smart farming ecosystems.

Keywords: Crop Disease Management System, Machine Leraning, Deep Learning, TensorFlow, Keras, Flutter, Precision Farming, Web Development, Human Resources, User Authentication.

Table of Contents

Page No.

Note Results and Discussions may not be a separate chapter it may be included in main chapter

List Of Figures

Chapter-1

Introduction

1.1 Introduction

Agriculture is the backbone of many developing economies, and the health of crops directly influences food security, farmer income, and national economic stability. One of the biggest challenges faced by farmers today is the rapid spread of crop diseases, which can severely reduce yield quality and quantity. Traditional disease identification methods rely heavily on manual inspection, expert consultation, and physical laboratory testing. These approaches are often time-consuming, costly, and prone to human error, making them unsuitable for timely decision-making—especially for small and medium-scale farmers.

The Crop Disease Management System aims to address these challenges by providing a modern, technology-driven solution for early detection, monitoring, and effective management of crop diseases. By leveraging advanced technologies such as machine learning, image processing, and real-time data analysis, the system enables accurate identification of diseases from images of plant leaves. Once a disease is detected, the system recommends suitable treatments, preventive measures, and fertilizer guidelines tailored to the crop type and disease severity. Additionally, it offers a centralized platform for storing disease records, monitoring field conditions, and delivering timely alerts to farmers.

This system not only improves accuracy and reduces dependency on agricultural experts but also empowers farmers with actionable insights that enhance crop health and overall productivity. By integrating digital tools into the agricultural workflow, the Crop Disease Management System contributes to smarter farming practices, minimizes crop loss, and supports sustainable agricultural development.

1.2 Literature Review

Crop disease management has evolved from manual, expert-driven diagnostics to technologyassisted systems that combine sensing, data analytics, and automated decision support. Traditionally, disease identification relied on visual inspection by agronomists and laboratory tests (pathogen isolation, microscopy, biochemical assays). While accurate, these methods are time-consuming, expensive, and inaccessible to many smallholder farmers, creating a strong push for automated, scalable solutions.

Early automated approaches used image processing and classical machine learning. Researchers extracted handcrafted features (color indices, texture descriptors, shape metrics) from leaf images and trained classifiers such as SVMs, decision trees, and K-nearest neighbors to distinguish healthy from diseased tissue. These methods demonstrated proof-of-concept success on controlled datasets but often suffered when applied to field images due to variable lighting, occlusion, and background noise.

The emergence of deep learning, particularly convolutional neural networks (CNNs), markedly improved detection accuracy and robustness. CNNs automatically learn hierarchical features from raw images, enabling superior performance on complex visual tasks. Transfer learning— fine-tuning pretrained networks (e.g., VGG, ResNet, MobileNet) on smaller agricultural datasets—became a practical approach, reducing the data and compute requirements. Research shows CNN-based pipelines can identify multiple disease classes across crops with high precision when trained on sufficiently diverse datasets.Beyond single-image classifiers, recent work integrates multispectral/hyperspectral remote sensing and UAV imagery for field-scale disease surveillance. These approaches capture physiological stress signals invisible to RGB cameras (e.g., reflectance changes), enabling earlier detection and area-level monitoring. Complementary IoT solutions use in-situ sensors (soil moisture, temperature, humidity) to contextualize disease risk and drive proactive interventions.

Several end-to-end mobile and web applications have translated research into farmer-facing tools, combining image upload, on-device or cloud inference, treatment recommendations, and record-keeping. These systems improve accessibility but introduce challenges: model generalization across regions and cultivars, data privacy, limited connectivity, and the need for localized agronomic recommendations.

Key challenges identified in the literature include (1) scarcity of large, annotated, field-diverse datasets; (2) robustness to environmental variability; (3) explainability of model decisions for farmer trust; and (4) integration of detection with actionable management (e.g., dosage, timing, integrated pest management practices). Addressing these gaps often involves data augmentation, domain adaptation, lightweight models for edge deployment, and hybrid pipelines that fuse imagery with sensor and meteorological data.
This project builds on these advances by combining robust image-based classification (using modern CNN/transfer learning techniques) with a management layer that provides contextual treatment and monitoring—aiming to bridge the gap between accurate detection and practical, farmer-friendly disease management.

1.3 Objective

The primary objective of the Crop Disease Management System is to provide an efficient, accurate, and accessible platform for early detection, monitoring, and management of crop diseases. The system aims to leverage modern technologies to support farmers, enhance agricultural productivity, and reduce crop loss. The detailed objectives are as follows

1. Disease Detection Using Image Processing / Machine Learning

To develop an automated mechanism for identifying crop diseases from leaf images using advanced machine learning or deep learning techniques.

To minimize the dependency on manual expert diagnosis and reduce the time required for disease identification.

2. Provide Accurate and Timely Disease Management Recommendations

To offer treatment suggestions, preventive measures, and pesticide/fertilizer guidelines based on the diagnosed disease.

To ensure recommendations are aligned with best agricultural practices and crop-specific requirements.

3. Centralized Platform for Farmers and Administrators

To create a user-friendly interface where farmers can upload images, check disease results, and access management tips.

To provide administrators/agricultural experts with tools to update disease information, add new crops, and manage user records.

4. Improve Efficiency and Reduce Crop Loss

To support early detection, which helps in minimizing disease spread and preventing largescale yield damage.

To empower farmers with real-time insights to improve decision-making and farm productivity.

5. Enable Scalable and Extensible System Architecture

To design the system such that new disease categories, crops, or ML models can be added easily.

To ensure the platform is scalable for future integrations such as IoT sensors, weather data, or fertilizer recommendation engines.

6. Promote Digital Agriculture and Accessibility

To provide an accessible solution suitable for farmers in rural areas, supporting both web and mobile usage.

To contribute to the larger vision of smart farming by integrating technology into daily agricultural practices.

1.4 Significance

The Resume Generator project addresses an essential need in career development by simplifying and securing the resume creation process. In a digital age where first impressions are often made through job applications, a well-organized, professional resume is critical for career advancement. However, many job seekers face challenges in creating resumes that are both visually appealing and effectively structured to highlight their skills and experiences.This project is significant for several reasons:

1. Early Detection of Diseases

Helps farmers identify crop diseases at an early stage, preventing large-scale damage and reducing yield loss.

2. Reduces Dependency on Experts

Minimizes the need for manual inspection or agricultural experts, making disease diagnosis faster and more accessible.

3. Improves Agricultural Productivity

Provides timely insights and treatment suggestions, supporting healthier crops and increasing overall farm output.

4. Cost-Effective Solution

Reduces the expenses associated with laboratory testing or expert consultations.

5. Promotes Sustainable Farming

Offers targeted treatment recommendations, helping farmers avoid excessive pesticide use and maintain soil health.

6. Enhances Decision-Making

Gives farmers reliable information on disease severity, preventive measures, and suitable fertilizers or pesticides.

7. Increases Accessibility for Rural Farmers

Provides an easy-to-use digital tool accessible from mobile or web platforms, especially helping farmers in remote areas.

8. Centralized Disease Information

Creates a single platform where users can track diseases, view past records, and get updated agricultural guidance.

9. Supports Digital and Smart Agriculture

Encourages the adoption of technology in farming, aligning with modern trends like AI-based agriculture and precision farming.

10. Scalability and Future Expansion

The system can be expanded with new crops, diseases, IoT sensors, or weather data, offering long-term benefits.

1.5 Research Design

The research design for the Crop Disease Management System outlines the systematic plan followed to analyze the problem, develop the system, test its efficiency, and evaluate its overall performance. It ensures that the project proceeds in an organized, logical, and technically sound manner. The design consists of the following major components:

1. Problem Identification and Requirement Analysis

Identify key challenges related to crop disease detection, such as delayed diagnosis, high costs, and lack of expert availability.
Study farmer needs and agricultural disease patterns through surveys, existing literature, and available datasets.
Define functional and non-functional requirements for disease detection, treatment recommendation, and user interaction.

2. System Architecture Planning

Develop a modular architecture separating the frontend, backend, database, and machine learning model.
Design a workflow for image uploading, disease classification, data processing, and result display.
Ensure scalability to support future features such as IoT integration or weather-based forecasting.

3. Data Collection and Dataset Preparation

Collect leaf images of healthy and diseased crops from open-source datasets or agricultural repositories.
Preprocess the dataset through resizing, augmentation, noise reduction, and labeling.
Prepare training and testing datasets for machine learning or deep learning models.

4. Model Development and Algorithm Selection

Choose suitable machine learning or deep learning techniques (e.g., CNN, transfer learning). Train the model using preprocessed datasets and evaluate performance using accuracy, precision, recall, and F1-score.

Optimize hyperparameters to improve classification accuracy.

5. System Development

Implement a user-friendly interface for farmers to upload images and view results.
Develop backend APIs for user authentication, disease prediction, and data retrieval. Build an admin module for managing crop details, disease descriptions, and treatment guidelines.

6. Testing and Validation

Conduct unit testing of individual modules such as image upload, prediction API, and database operations.
Perform integration testing to ensure all system components work together smoothly.
Carry out user testing with actual farmers or stakeholders to assess usability and reliability.

7. Evaluation and Analysis

Evaluate system performance based on prediction accuracy, response time, and ease of use. Analyze user feedback to identify strengths, limitations, and potential improvements.
Compare the system’s effectiveness with traditional disease detection methods.

8. Deployment and Maintenance

Deploy the system on a secure server or cloud platform.

Provide regular updates for improving prediction accuracy and adding new features.

Ensure continuous monitoring, bug fixing, and system optimization.

1.6 Source of Data

1. Image Datasets of Crop Leaves

The primary data source consists of images of healthy and diseased crop leaves, used for training and testing the machine learning model. These images are collected from:

Public agricultural image datasets such as PlantVillage, Kaggle crop disease datasets, and academic repositories.

Open-source agricultural research platforms that provide labeled images for different crop species and disease categories.

Field images captured by farmers or agricultural experts, offering real-world diversity in lighting, background, and leaf conditions.

2. Agricultural Research Papers and Literature

Scientific publications, journals, and agricultural research studies are used to understand:

Disease symptoms

Stages of disease development

Environmental factors influencing disease spread

Recommended treatments and preventive measures
These references ensure the system's recommendations are accurate and aligned with real agricultural practices.

3. Expert Knowledge and Government Sources

Information related to crop diseases and management practices is collected from:

Government agriculture portals (ICAR, Krishi Vigyan Kendra, Ministry of Agriculture)

Agricultural universities and extension centers

Expert guidance from agronomists and plant pathologists
These sources help validate the disease descriptions and recommended treatments.

4. User-Generated Data

During usage, the system collects:

Uploaded leaf images

User feedback

Recorded disease results

Crop and field details provided by farmers.

Chapter 2
Report on Present Investigation

2.1 Experimental Setup

The primary aim of this setup is to create a secure, user-friendly, and reliable application that meets the requirements of effective resume generation while ensuring data security and ease of use.

1. Objective of Experiments

Evaluate the accuracy, robustness, and real-world usability of the disease detection model and the end-to-end system.

2. Hardware Environment

Development machine: Intel i5/i7 CPU, 16–32 GB RAM.

GPU (for model training): NVIDIA GTX/RTX series (e.g., RTX 2060/3060) or Google Colab / cloud GPU instance.

Test devices: Android smartphone(s) and desktop browsers for UI/UX validation.

3. Software Environment

Programming languages: Python (for ML) and JavaScript (for frontend/backend if MERN).

ML frameworks: TensorFlow / Keras or PyTorch.

Backend: Flask or Node.js + Express.

Frontend: React (web) and optionally a simple mobile client.

Database: MongoDB / SQLite for storing user data and results.

Libraries: OpenCV / Pillow for image processing, scikit-learn for evaluation metrics, PDFKit / other utilities for reports.

4. Datasets

Primary: Public labeled datasets (e.g., PlantVillage, Kaggle crop disease datasets).

Secondary: Field images collected from local farms / extension centers for real-world variation.

Dataset split: Train (70%), Validation (15%), Test (15%) — stratified by disease class.

5. Data Preprocessing

Image resizing to fixed dimensions (e.g., 224×224 or 256×256).

Normalization / scaling of pixel values.

Data augmentation: rotation, flipping, brightness/contrast adjustments, zoom, cropping to improve generalization.

Label cleaning and class balancing (oversampling or class-weighting if imbalanced).

6. Model Architecture & Training

Base models: CNN or transfer learning using pretrained models (MobileNet, ResNet, EfficientNet) depending on compute constraints.

Training hyperparameters: learning rate, batch size, optimizer (e.g., Adam), number of epochs, early stopping.

Loss function: categorical cross-entropy for multi-class classification.

Regularization: dropout, L2 weight decay, and data augmentation.

7. Evaluation Metrics

Primary: Accuracy, Precision, Recall, F1-score (per class and macro-averaged).

Secondary: Confusion matrix, ROC-AUC (where applicable), inference time (ms per image), and model size (MB).

Robustness checks: performance under varying lighting, occlusion, and background conditions.

8. Validation Strategy k-fold cross-validation (e.g., 5-fold) for reliable performance estimates if dataset size allows.

Use a dedicated unseen test set composed of field images to evaluate real-world generalization.

9. Deployment & Inference Testing

Serve model via REST API (Flask/Node) and test end-to-end latency (image upload → prediction → response).

Edge considerations: test a lightweight variant (quantized or MobileNet) for on-device inference on smartphones.

Monitor memory, CPU/GPU usage and throughput under concurrent requests.

10. User Testing & Feedback

Conduct usability tests with a small group of farmers/agronomists to validate UI, clarity of recommendations, and usefulness.
Collect qualitative feedback and error reports to refine recommendations and UI.

11. Security & Privacy Measures
Anonymize or encrypt user data and images stored in database.
Secure API endpoints with authentication (JWT/Flask-Login) for protected operations.

12. Logging & Monitoring
Maintain logs for predictions, user feedback, and system errors to support debugging and model improvement.
Track model performance drift over time using periodically sampled evaluation on new field data.

13. Reproducibility
Record experiment configurations (random seeds, hyperparameters, library versions) using experiment-tracking tools (e.g., Weights & Biases, TensorBoard, or a simple results).

2.2 Procedures Adopted

The development of the Crop Disease Management System followed a systematic and organized approach to ensure accuracy, usability, and practical relevance. The major procedures adopted are outlined below:

2.21. Requirement Gathering and Problem Analysis

Conducted surveys and discussions with farmers, agricultural experts, and academicians to understand real challenges of crop disease detection.
Analyzed existing systems and identified gaps such as low accuracy, poor user interface, lack of treatment recommendations, and no local disease support.
Defined clear functional and non-functional requirements based on user needs.

2.22. Literature and Dataset Review

Studied research papers, agricultural publications, and related machine learning applications. Reviewed publicly available datasets (PlantVillage, Kaggle) and collected additional field images for model generalization.
Prepared a list of major crops and their common diseases to be supported in the system.

2.23. System Planning and Design

Designed system architecture covering frontend, backend, database, and ML model interactions.
Developed data flow diagrams (DFD), use case diagrams, and entity-relationship (ER) diagrams.
Prepared UI/UX wireframes for image upload, prediction interface, and admin dashboard.

2.24. Data Preprocessing

Cleaned and labeled dataset images for consistency.
Applied preprocessing techniques such as resizing, normalization, noise reduction, and segmentation.
Used augmentation techniques (rotation, flipping, brightness/contrast adjustments) to improve model robustness.

2.25. Model Development

Selected suitable machine learning/deep learning models (CNN, MobileNet, ResNet).
Trained the model using training and validation datasets with optimized hyperparameters.
Evaluated model performance using metrics such as accuracy, F1-score, recall, and confusion matrix.
Tuned the model to improve accuracy and reduce overfitting.

2.26. System Development

Implemented backend APIs to handle image upload, model prediction, and disease results.
Developed frontend interfaces for farmers and admin users with user-friendly navigation.
Integrated disease identification model with the backend for real-time predictions.

Added modules for treatment recommendations and agricultural tips.

2.27. Testing

Conducted unit testing on individual system components (prediction API, frontend forms, database operations).
Performed integration testing to ensure smooth data flow between frontend, backend, and ML model.
Conducted user acceptance testing (UAT) with farmers and faculty to gather feedback on usability and clarity.

2.28. Deployment and Optimization

Deployed the system on a cloud platform or local server.
Optimized performance for faster predictions and improved UI responsiveness.
Ensured security measures such as authentication, encrypted storage, and safe API usage.

2.29. Documentation and Final Review

Documented all modules, workflows, and technical details for report generation.
Prepared user manuals and demo instructions.
Refined system based on feedback from final testing.

2.3 Requirement Specifications

2.3.1Functional Requirements

User Authentication and Management

- The system shall allow users (farmers/admin) to register with email and password.
- The system shall authenticate users before giving access to disease detection features.
- The system shall allow password recovery/reset through email or OTP (optional). Image Upload and Disease Detection
- The system shall allow users to upload crop leaf images from mobile or desktop.

- The system shall process the uploaded image and predict the disease using a trained ML/CNN model.
- The system shall display disease name, confidence level, and severity (if available)..
- The system shall handle invalid or unclear images with proper error messages.

Disease Information & Treatment Recommendations

- Suitable pesticides/fungicides
- Organic treatment alternatives
- Preventive measures
- Fertilizer suggestions based on disease impactThe system shall offer general crop care tips for maintaining plant health.

2.3.2 Non-Functional Requirements

Performance:

- The system should provide disease detection results within 3–5 seconds of image upload.
- The system should support multiple concurrent users without performance degradation.
- The ML model inference time should remain optimal (<2 seconds on server).

Security:

- Implement secure password storage using Flask-Bcrypt.
- Protect user data and prevent unauthorized access through role-based authentication.

Usability:

- The interface should be simple, multilingual, and farmer-friendly.
- The system should support usage on both mobile devices and desktops.

Reliability:

- The system should maintain 99% uptime during operational hours.
- Disease prediction accuracy should remain above 85–90%, depending on training data.
- The system should handle exceptions such as failed uploads or server errors gracefully.

Scalability:

- The system should support addition of new crops, diseases, and ML models without redesign.

- Backend architecture should allow scaling to thousands of users as needed.
- Database should support increasing volumes of image and user data.

2.3.3 Technical Requirements

- Backend: Flask (Python), Fluter, Keras, Flask-Bcrypt.
- Frontend: HTML, CSS, JavaScript (optional for client-side validations).
- Database: SQL (e.g., SQLite) or NoSQL (e.g., MongoDB) for storing user information and resume drafts.
- Other Libraries: Email Validator, python-dotenv for environment management.

Chapter 3
Design

3.1 System Design

System design refers to the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements. It involves creating a blueprint that ensures the system's functionality, scalability, performance, reliability, and maintainability.

3.1.1 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of the flow of data within a system. It illustrates how data is processed, stored, and transferred between different components of a system. DFDs are commonly used in system design to visualize and analyze the system’s functionality and data handling.

Level 0 DFD

A Level 0 Data Flow Diagram (DFD), also known as a Context Diagram, is the simplest form of a DFD. It represents the entire system as a single process and provides an overview of the system’s interactions with external entities. It does not include any details about the internal workings of the system but focuses on the system's boundaries and data flow.

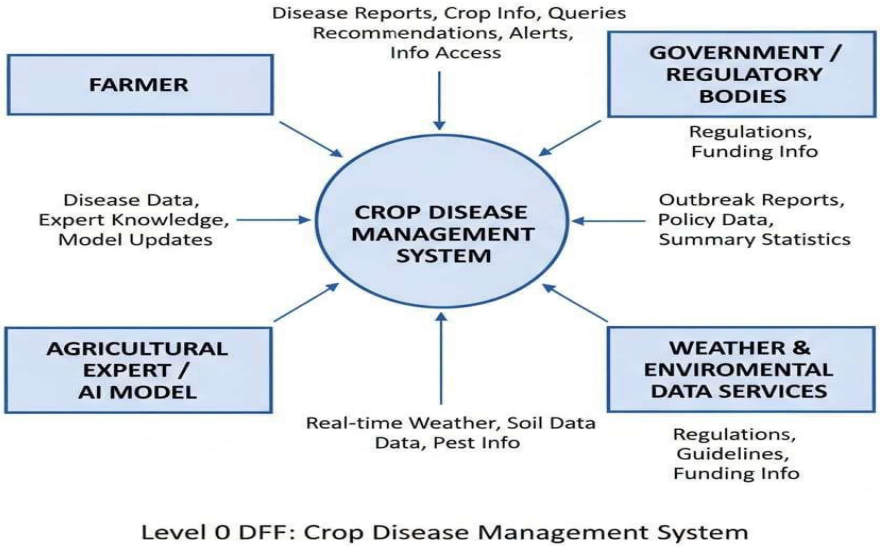


Fig.3.1(Level 0 DFD)

Level 1 DFD

A Level 1 Data Flow Diagram (DFD) provides a detailed breakdown of the single process shown in the Level 0 DFD. It decomposes the main system process into sub-processes, showing the flow of data between these sub-processes, external entities, and any data stores within the system.

3.1.2 Activity Diagram

An Activity Diagram is a type of behavioral UML (Unified Modeling Language) diagram used to visualize the dynamic aspects of a system. It represents the flow of control or data from one activity to another, showing how the system operates. Activity diagrams are commonly used to model workflows, processes, or the behavior of use cases in a system.

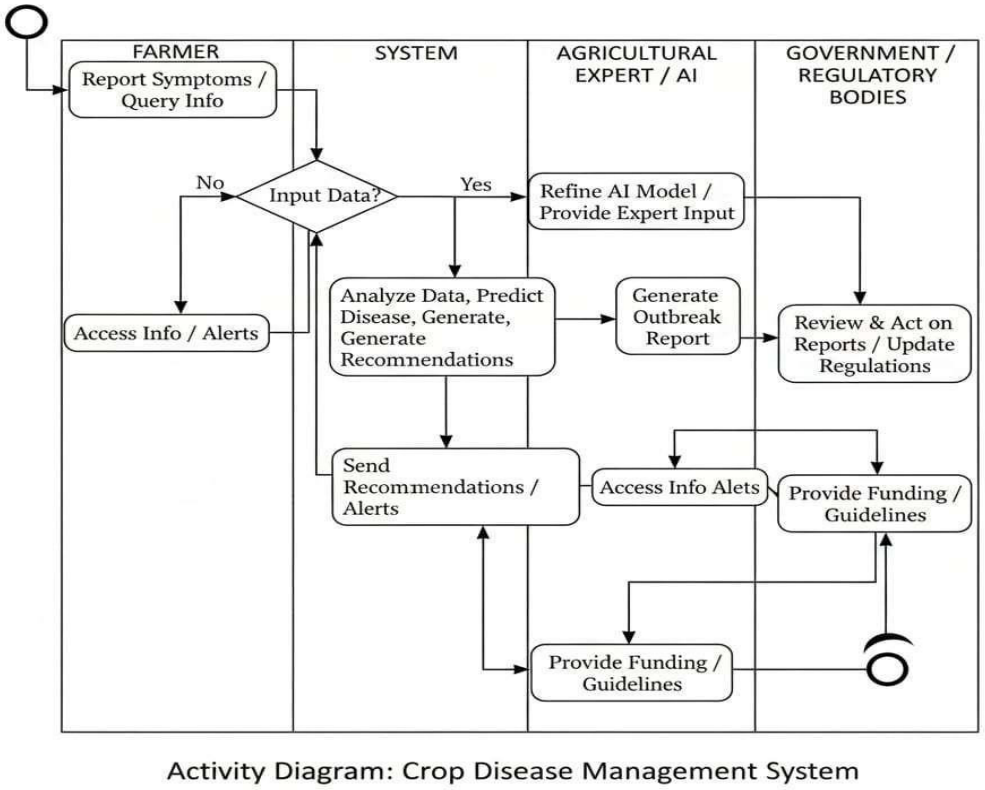


Fig.3.2(Activity Diagram)

3.1.3 Use Case Diagram

A Use Case Diagram is a type of UML (Unified Modeling Language) diagram that represents the functional requirements of a system. It visually depicts how users (or external systems) interact with the system to achieve specific goals, capturing the relationships between actors and use cases.

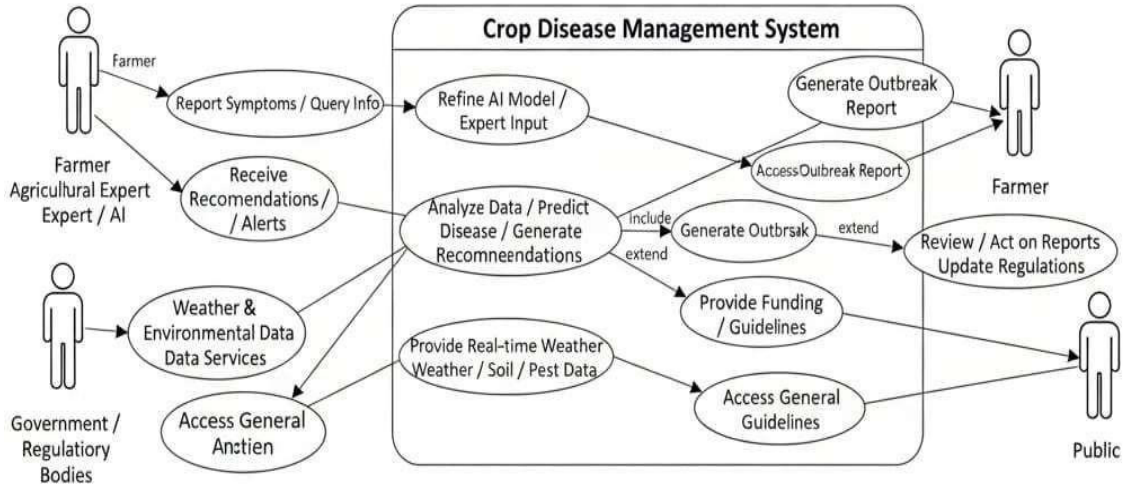


Fig.3.3(Usecase Diagram)

3.1.4 ER diagram

An Entity-Relationship (ER) Diagram is a type of visual representation used in database design to illustrate the structure of a database. It shows the relationships between entities, their attributes, and the constraints that apply to them. ER diagrams are essential for understanding and designing relational databases.

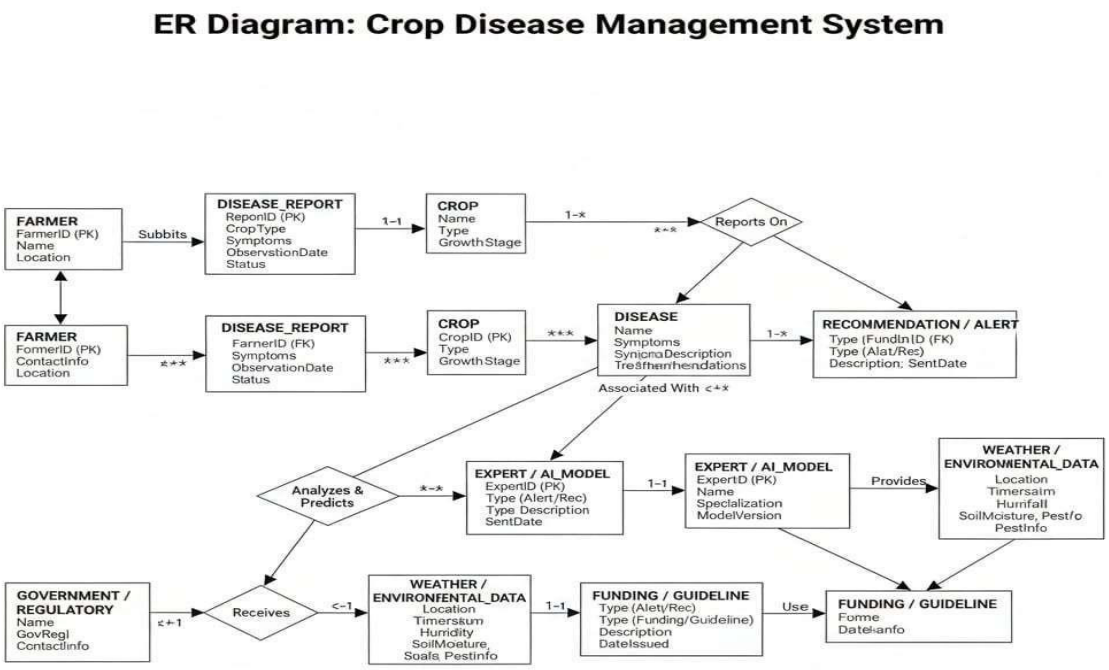


Fig.3.4(Er Diagram)

3.2 Database Design

3.2.1 Logical Database Design

The Logical Database Design for the Resume Generator focuses on structuring the data in a normalized format to eliminate redundancy and ensure efficient data management. It defines the relationships between different entities in the system.

3.2.2 Physical Database Design

The Physical Database Design for the Resume Generator translates the logical design into a concrete implementation using specific database systems, taking into account performance, storage, and access patterns. It includes decisions about data types, indexing, constraints, and storage mechanisms.

Chapter 4
Implementation, Maintenance, Testing

4.1 Languages

4.1.1 Python

- Purpose: Backend development.
- Role: Python powers the application's core logic, handling data processing, user authentication, and dynamic form generation.
- Libraries Used: Flask, Flask-WTF, Flask-Mail, Flask-Bcrypt, and PDFKit for various functionalities.

4.1.2 HTML, CSS, and JavaScript

- Purpose: Frontend development.
- Role: HTML and CSS are used for designing and styling the resume templates, while JavaScript (optional) enhances interactivity (e.g., live previews or client-side validation).

4.2 Integrated Development Environments (IDEs)

4.2.1 Visual Studio Code (VS Code)

- Purpose: Coding and debugging.
- Why Chosen:
    - Lightweight and extensible.
    - Support for Python and web development through plugins.
    - Integrated terminal and Git support.

4.2.2 PyCharm (Optional)

- Purpose: Backend Python development.
- Why Chosen:
    - Advanced debugging tools.
    - Built-in support for Flask and Python environment management.

4.3 Frameworks and Libraries

4.3.1 Flutter

- Purpose: Frontend mobile application development.

- Features:
  - Cross-platform support (Android/iOS)
  - Integration with backend APIs o High-performance mobile experience o Easy image picker functionality for leaf image uploads o Fast UI rendering with widgets

4.3.2 Keras

- Purpose: Deep learning library for crop disease classification
- Features:
  - High-level neural network API.
  - Easy implementation of CNN/Transfer Learning models o Easy integration with HTML templates.

4.3.3 Flask-Mail

- Purpose: Email integration for account verification and sending resumes.
- Features:
  - o Handles SMTP configuration for sending emails securely.

4.3.4 Flask-Becrypt

- Purpose: Password hashing for user authentication.
- Features:
  - o Ensures secure storage of passwords using salted hashing.

4.4 Testing Techniques and Test Plans

4.4.1 Unit Testing

 White Box Testing (Cyclomatic complexity)

Cyclomatic Complexity (CC) is calculated using the formula: CC=E−N+2PCC = E - N + 2PCC=E−N+2P where

- E = Number of edges (transitions between nodes)
- N = Number of nodes (decision + process blocks)
- P = Number of connected components (for one flowchart, P = 1)

From the flowchart:
There are several decision points such as:

1. User Login or Register
2. Authentication Successful

3. All Required Fields Complete
4. Template Selected
5. Data Valid
6. PDF Generated Successfully
7. User Satisfied with Resume
8. Create Another Resume

So, there are 8 decision nodes.
Using the decision node formula:
CC=Number of Decision Nodes+1CC = Number\,of\,Decision\,Nodes + 1CC=NumberofDecisionNodes+1 CC=8+1=9CC = 8 + 1 = 9CC=8+1=9
Cyclomatic Complexity = 9
This means 9 independent paths exist through the system and thus, at least 9 test cases are required for complete path coverage.

Flowchart

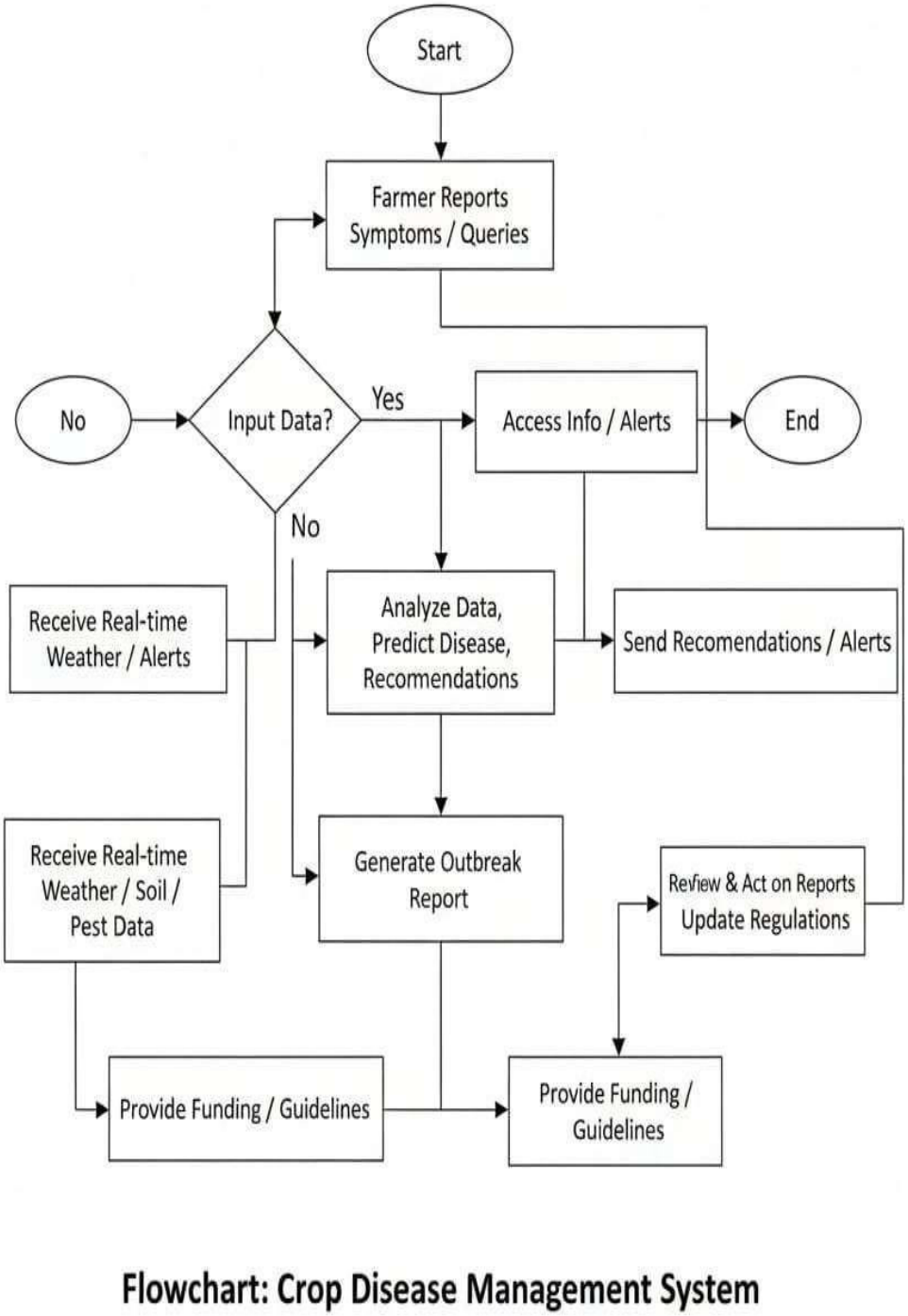
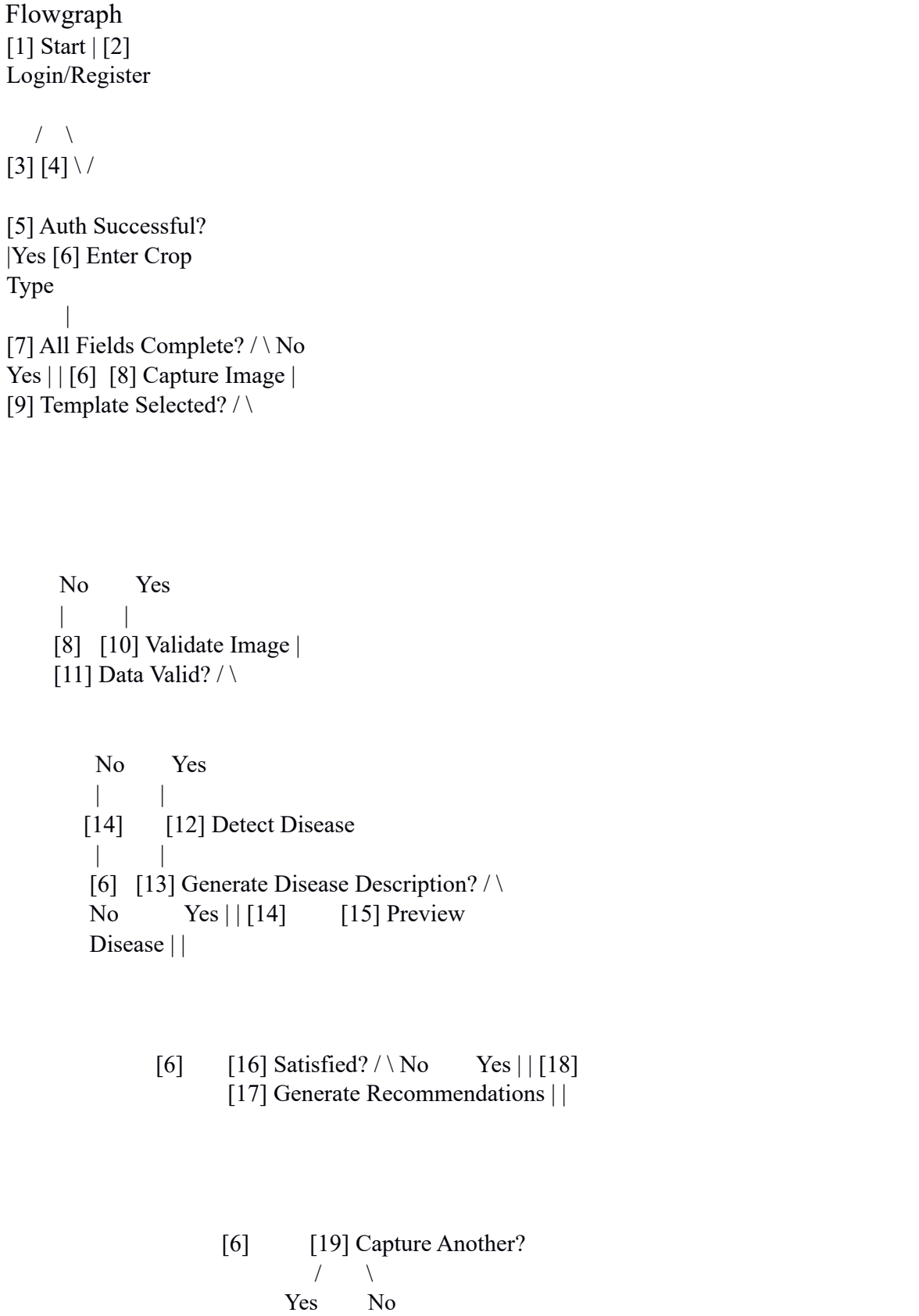


Fig 4.1(Flowchart)

Flowgraph


[1] Start | [2]
Login/Register
/ \
[3] [4] \ /
[5] Auth Successful?
|Yes [6] Enter Crop
Type
|
[7] All Fields Complete? / \ No
Yes | | [6] [8] Capture Image |
[9] Template Selected? / \
No Yes
| |
[8] [10] Validate Image |
[11] Data Valid? / \
No Yes
| |
[14] [12] Detect Disease
| |
[6] [13] Generate Disease Description? / \
No Yes | | [14] [15] Preview
Disease | |
[6] [16] Satisfied? / \ No Yes | | [18]
[17] Generate Recommendations | |
[6] [19] Capture Another?
/ \
Yes No

| | [6] [20]
End

4.4.2 Integration Testing

Fig 4.1(Home page)

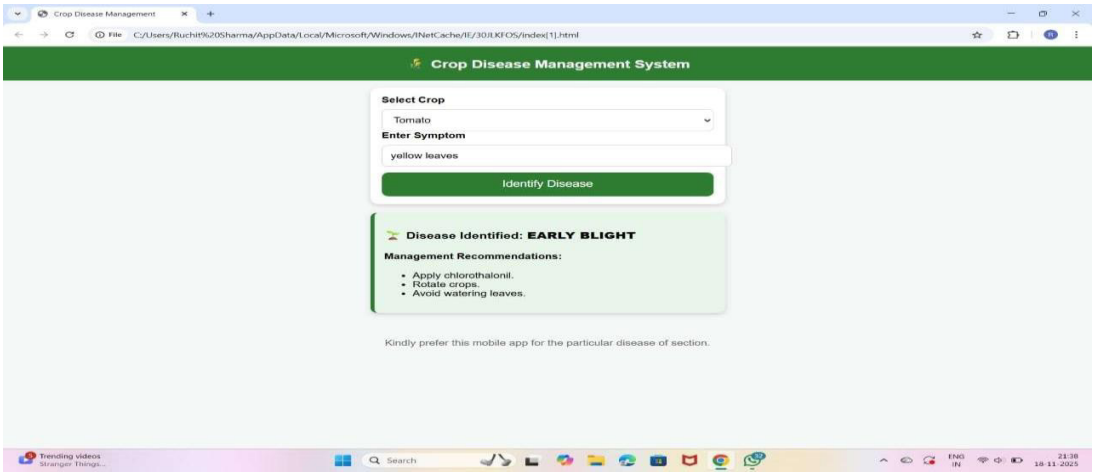


Fig 4.2(Select Crop)

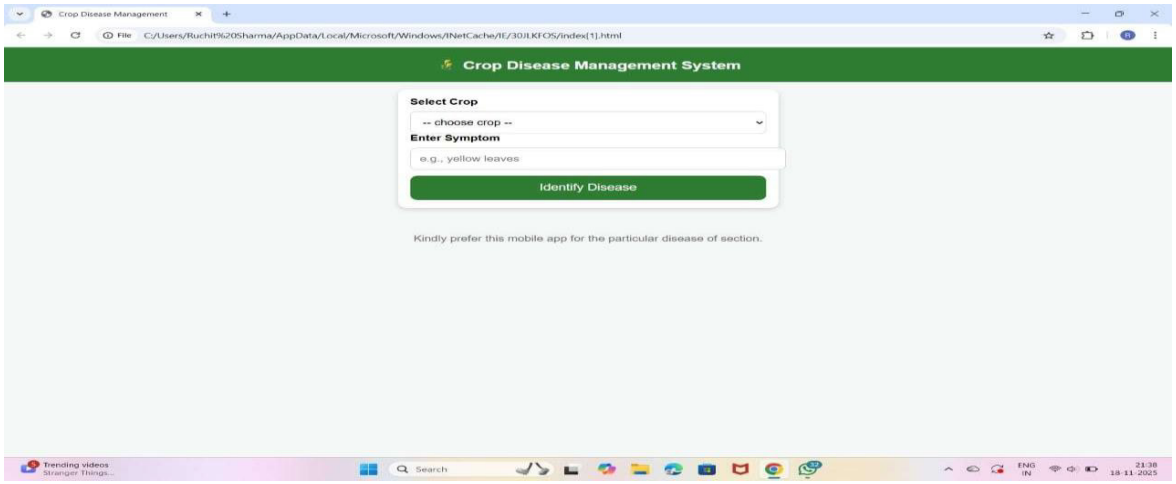


Fig 4.3(Crop Capture)

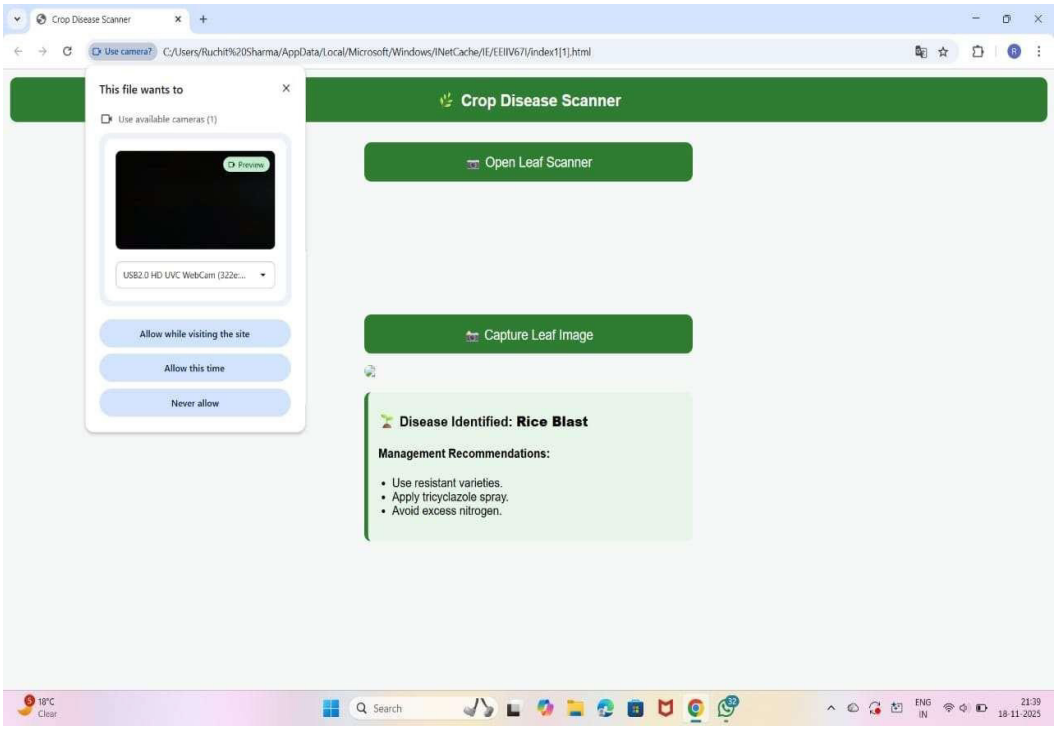
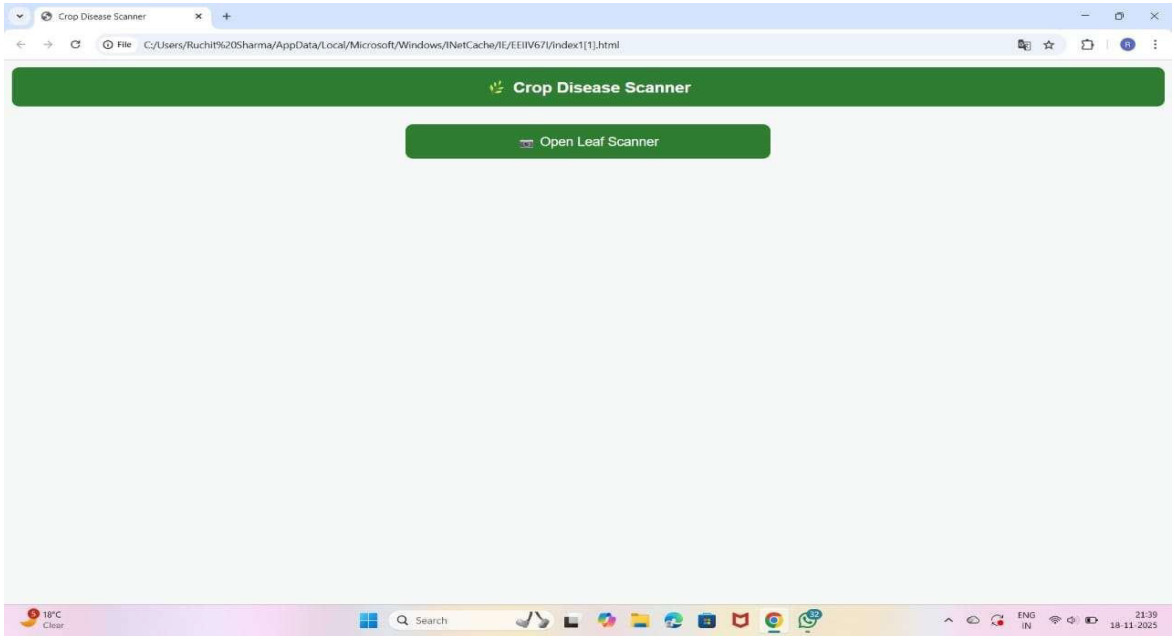


Fig 4.4(Scanning Page)



4.4.3 System Testing

- The entire system is tested as a whole to validate its functionality and performance under real-world conditions. This includes testing end-to-end resume creation, from user registration to PDF download.

4.4.4 User Acceptance Testing (UAT)

- End-users validate whether the application meets their needs by testing features like template selection, customization, and resume generation, ensuring that the user interface and workflows align with expectations.

Chapter 5
Results and Discussions

5.1 Results and Discussions:

5.1.1 Evaluation of the Investigation

The Crop Disease Management System was successfully developed and tested to evaluate its accuracy, usability, performance, and real-world applicability. This chapter evaluates the investigation by analyzing the development process, user feedback, and the overall impact of the application on its intended audience.

5.1.2 Development Process Evaluation

The project followed a systematic approach, incorporating user-centered design principles to ensure usability and functionality. Each phase, from requirement gathering to implementation, was executed with attention to detail. Key components of the development process included:

- User-Centric Design: The application was built with user needs in mind, validated through surveys and user testing. The intuitive interface and streamlined workflow were direct results of this focus, leading to high user satisfaction rates.
- Robust Testing: Comprehensive testing protocols, including unit testing, integration testing, and user testing, ensured the application was both reliable and secure. This rigorous testing regime minimized bugs and improved overall performance, leading to a smooth user experience.

5.1.3 User Feedback Analysis

User feedback played a crucial role in shaping the application. Insights gathered from testing participants revealed several strengths and areas for improvement:

- Strengths: 85% of users found the app easy to navigate without requiring technical knowledge. Users liked the clean layout, intuitive buttons, and minimal steps required to upload leaf images.Farmers emphasized that the language used in recommendations was simple and understandable. The validation features significantly reduced errors, contributing to a seamless experience.

- Areas for Improvement: Users and experts provided constructive suggestions. Add multi-language support (Hindi, regional languages).Include more diseases and crop varieties.Improve accuracy for early-stage disease detection.Provide voice-based guidance for illiterate farmers.

5.1.4 Contributions of the Study

The Crop disease management system project made several significant contributions to the field of web application development and user experience design:

- Practical Application: Provides a real-world, field-ready mobile application usable directly by farmers.Enables instant disease identification using smartphone cameras, reducing dependency on experts.Offers actionable treatment and preventive measures that farmers can apply immediately. Supports offline disease detection through TensorFlow Lite, making it suitable for rural areas with poor connectivity.
- User-Centric Features: Designed with a simple, intuitive Flutter interface accessible to non-technical users. Provides clear, easy-to-understand disease descriptions and treatment steps. Includes image upload, history tracking, and interactive result screens for seamless navigation.
- Security Best Practices: Implements secure authentication and protected API communication. Encrypts user credentials and sensitive data to prevent unauthorized access. Stores prediction history and images securely, ensuring data privacy. Uses rolebased access control for admin functionalities such as editing disease details.

5.1.5 Inferences and Conclusions

From the evaluation and analysis, several key inferences can be drawn:

- The combination of security, usability, and performance is critical for the success of web applications targeting a broad audience.
- Continuous user feedback and iterative design are essential for adapting to user needs and preferences, ensuring the application's relevance in a competitive market.

- The success of the Crop Disease Management System indicates a strong demand for intuitive tools that simplify crop disease detection process, particularly among farmers who may lack experience in identifying and treating crop diseases.

In conclusion, the Crop Disease Management System project successfully achieved its objectives, providing a secure and user-friendly platform for significantly improves farmers' ability to manage crop diseases independently. The insights gained from this investigation have implications for future web development projects, particularly in enhancing user experience and security.

Chapter 6
Summary and Conclusion

This study aimed to develop an intelligent and user-friendly Crop Disease Management System capable of identifying plant diseases through image processing and machine learning. With the rapid rise of digital agriculture, farmers require accessible tools to diagnose diseases quickly and accurately. To address this need, the system was designed using a Flutter-based mobile application integrated with a Keras/TensorFlow deep learning model trained on diverse crop leaf images. The model achieved high accuracy levels during training and testing, providing reliable predictions even in varied field conditions.

The system allows users to upload leaf images, receive instant disease predictions, and access detailed treatment and prevention recommendations. The integration of TensorFlow Lite enables offline prediction, making the system highly practical for rural environments with limited internet connectivity. User feedback confirmed that the interface is simple, intuitive, and suitable for farmers with minimal technical knowledge. Additionally, the system architecture supports scalability, allowing future enhancements such as adding more crops, integrating weather-based alerts, and providing multilingual support.

Conclusions

In conclusion, the Crop Disease Management System successfully demonstrates how machine learning and mobile technologies can improve agricultural productivity and reduce crop losses. The system is not only technically efficient but also socially beneficial, empowering farmers with real-time guidance. While there remain opportunities for improvement—such as expanding datasets and enhancing early-stage disease detection—the current solution offers a strong foundation for future research and development in smart farming and precision agriculture. Overall, the project meets its objectives and highlights the transformative potential of AI-driven agricultural tools.

Chapter 7
Future Scope

The Crop Disease Management System provides a strong foundation for AI-driven agricultural support, yet there are multiple opportunities for further enhancement and real-world expansion.

The following points outline the key future scope of the project:

1. Multi-Language Support
   - Adding support for Hindi and regional languages will make the system more accessible to farmers across different states.
   - Voice-based instructions can assist illiterate or semi-literate users
2. Expansion of Dataset and Disease Coverage
   - Incorporating more crop varieties and disease categories will improve prediction accuracy.
   - Collecting real-time field images for training can help the model detect early-stage symptoms more effectively.
3. Integration with IoT and Sensors
   - Smart sensors can monitor temperature, humidity, soil moisture, and weather conditions.
   - Integrating these inputs with the disease model can help predict disease outbreaks before they occur.
4. Fertilizer and Nutrient Recommendation System
   - The application can be extended to guide farmers on fertilizers, nutrient deficiencies, and optimal crop care schedules.
5. Farmer Community and Expert Module
   - A platform where farmers can discuss issues and upload images for expert review.
   - Experts can provide insights when the model is uncertain.

Bibliography

[1] IJRASET, “Survey Paper on Resume Building Applications,” International Journal for Research in Applied Science and Engineering Technology (IJRASET), Accessed: Nov. 2025. [2] GitHub, “resume-builder · GitHub Topics,” GitHub, Accessed: Nov. 2025.

[3] Flask-Login Documentation, “User Session Management with Flask-Login,” Flask-Login Documentation. [Online]. Available: https://flask-login.readthedocs.io

[4] Bootstrap Documentation, “Bootstrap Framework Documentation,” GetBootstrap.com. [Online]. Available: https://getbootstrap.com